

# VetLab

---

Consola de agenda veterinaria construida con  
React 19: Del requerimiento funcional al  
dominio técnico.

**PROYECTO:** Proyecto Final Pyxis

**STATUS:** Production Ready Build

React 19

TypeScript Strict

Vite

CSS Modules

Suspense & Actions



# El problema que resuelve

VETLAB ENGINEERING REPORT / 2024

## Actores del Sistema



**Recepción**  
Usuario principal de la consola operativa.



**Mascotas**  
Registros clínicos y dueños asociados.



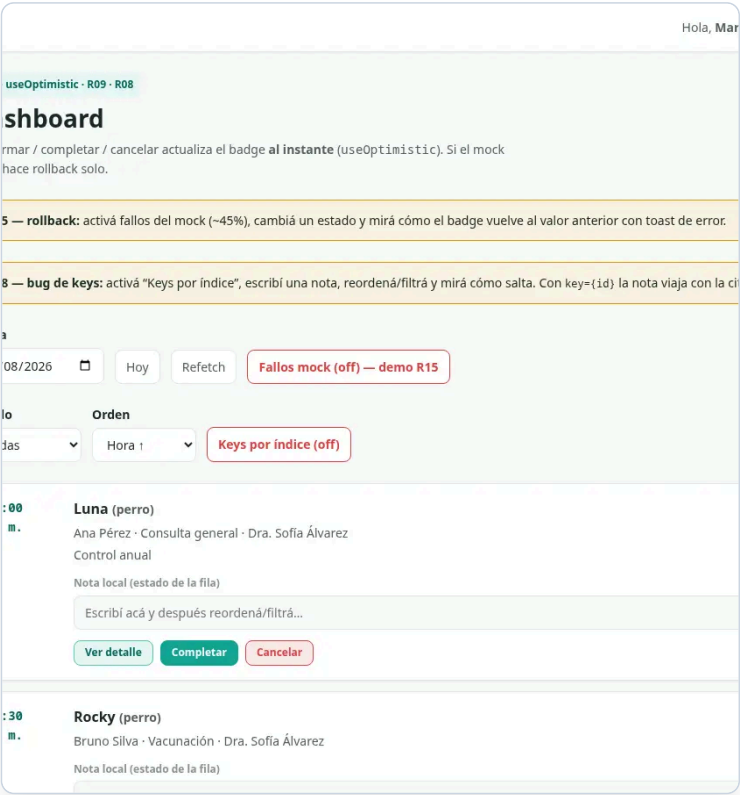
**Veterinarios**  
Catálogo de servicios y disponibilidad.

## Flujos Críticos

- Operación diaria desde un dashboard centralizado y reactivo.
- Reserva de citas validada contra disponibilidad real en tiempo real.
- Gestión de historiales consistentes para decisiones clínicas precisas.

### REGLA DE NEGOCIO CENTRAL

Dos citas activas del mismo veterinario no pueden ocupar intervalos superpuestos. Una cita cancelada debe liberar el horario inmediatamente.



Interfaz de gestión operativa: Dashboard de control diario.

# Alcance y Stack Técnico

VETLAB ENGINEERING REPORT / 2024

## Núcleo Tecnológico



**React**

v19.2.7 (Latest Core)



**TypeScript**

Strict Mode Configuration



**Vite**

v8.1.1 (Build Tool)



**React Router**

v7.18.1 (Vite Plugin)



**CSS Modules**

Scoping & Design Tokens



**Mock API**

Promises & Memory Cache

Zona horaria de negocio: America/Montevideo



### Alcance Deliberado

- Arquitectura SPA **Client-only** sin backend persistente.
- Persistencia de datos restringida a la **memoria del navegador** .
- Uso de **Promises con latencia simulada** para flujos asíncronos reales.
- Sin dependencias de librerías externas para el **estado global** .



### Objetivo de Aprendizaje

Concentrarse en los mecanismos fundamentales de **React 19 Core** (Suspense, Actions, Hooks) antes de delegar responsabilidades en frameworks o librerías de terceros.

# Arquitectura de la Solución

VETLAB ENGINEERING REPORT / 2026

01 Rutas y Layout

02 Vistas y Componentes

03 Hooks, Contexts y Actions

04 Mock API y Reglas de Negocio

05 Modelos de Dominio y Seed

## DOMINIO Y DATOS

`domain/models.ts` Contratos de dominio y tipos derivados de TypeScript.

`data/mockApi.ts` Promesas, latencia, cache y reglas de solapamiento de citas.

## LÓGICA Y ESTADO

`hooks/` Reutilización de lógica de datos y efectos secundarios.

`context/` & `actions/` Estado global (Sesión, Tema) y mutaciones de formulario.

## PRESENTACIÓN Y ESTILO

`components/` & `views/` Unidades atómicas, modales accesibles y orquestación.

`styles/tokens.css` Lenguaje visual centralizado y theming dinámico.

✓ Responsabilidad Única

✓ Separación de Intereses

✓ Dependencias al Dominio

✓ Composición (DRY)

# Del estado local a flujos predecibles

VETLAB ENGINEERING REPORT / 2024



R01

## useState: Gestión de UI

- ✓ Formularios controlados con validación derivada en tiempo real.
- ✓ Distinción clara entre estado real y datos calculables.
- ✓ Manejo de estados booleanos simples (loading, error, success).



R02

## useEffect: Sincronización

- ✓ Carga de citas por fecha con dependencias explícitas.
- ✓ Cleanup para evitar actualizaciones en componentes desmontados.
- ✓ Control de race conditions en peticiones asíncronas.



R03

## useRef: Persistencia Muda

- ✓ Foco imperativo en inputs al abrir modales o tras errores.
- ✓ Almacenamiento de valores que no disparan re-renders.
- ✓ Acceso directo al DOM para integraciones de terceros.



R04

## useReducer: Lógica de Flujo

- ✓ Wizard de alta para dueño y mascota con estados interdependientes.
- ✓ Acciones tipadas y transiciones de estado predecibles.
- ✓ Extracción de la lógica de negocio fuera del componente visual.



**Aprendizaje clave:** La elección del hook se basa en la **forma del problema**. useState es para estado simple, mientras que useReducer escala mejor ante flujos con múltiples transiciones relacionadas.

# Reutilización, Contexto y Performance

VETLAB ENGINEERING REPORT / 2024



## Context API

- ✓ **SessionContext** : Gestión global del staff y autenticación simulada.
- ✓ **ThemeContext** : Soporte nativo para modo claro/oscuro mediante tokens.
- ✓ Encapsulación total mediante hooks personalizados como `useSession()` .
- ✓ Memoización del valor del Provider para evitar propagación de renders innecesarios.

---

*Propósito: Evitar el prop-drilling y centralizar la identidad de la aplicación.*



## Hooks Propios

- ✓ **useCitas(date)** : Orquestación de carga y actualización de agenda.
- ✓ **useDisponibilidad** : Lógica de validación contra reglas de negocio asíncronas.
- ✓ Interfaz estándar: **data, loading, error** y función de refetch.
- ✓ Lógica de **cleanup anti-race** centralizada para evitar fugas de memoria.

---

*Propósito: Desacoplar la lógica de datos de la representación visual.*



## Performance

- ✓ **useMemo** : Derivación pesada del modelo de grilla de slots por veterinario.
- ✓ **useCallback** : Estabilización de handlers pasados a componentes hijos.
- ✓ **React.memo** : Prevención de repintado en celdas de agenda con props estables.
- ✓ Medición del impacto: Optimización crítica en vistas con alta densidad de nodos.

---

*Aprendizaje: "memo" solo es efectivo cuando las referencias son estables.*

# Identidad, portales y aislamiento de fallos

VETLAB ENGINEERING REPORT / ROBUSTEZ UI

## Gestión de Keys

Uso de identificadores estables para garantizar la integridad del DOM durante filtrados y reordenamientos de la agenda.

*Evidencia: Demostración reproducible del bug `key={index}` que causa pérdida de estado en inputs durante el filtrado de veterinarios.*

IDENTIDAD ESTABLE    RECONCILIACIÓN

## React Portals

Renderizado de modales y toasts fuera del contenedor visual principal, manteniendo el árbol lógico y el acceso al Contexto.

*Accesibilidad: Gestión de tecla `Escape`, foco inicial automático y restauración del foco al cerrar el modal de citas.*

DOM AISLADO    A11Y

## Error Boundaries

Implementación de límites de error granulares para evitar que fallos en secciones secundarias derriben toda la aplicación.

*Resiliencia: Un fallo en la carga del historial de una mascota muestra un fallback específico con opción de reintento local.*

AISLAMIENTO    GRACEFUL DEGRADATION

### APRENDIZAJE CLAVE:

La identidad de elementos, la ubicación del DOM y los límites de error son **decisiones arquitectónicas**, no simples detalles de implementación de la interfaz.

# Concurrencia, carga progresiva y Suspense

VETLAB ENGINEERING REPORT / 2024

## R11 — useTransition

### Búsquedas fluidas sin bloqueo

Entrada del buscador: actualización **urgente**  
Filtrado de resultados: transición **no urgente**  
`isPending` comunica el estado al usuario  
UI responde de inmediato mientras el cómputo se difiere

 Prioridad diferenciada: **urgent vs. deferrable**

## R12 — lazy + Suspense

### Code-splitting bajo demanda

Vista de Reportes cargada solo cuando se accede  
Fallback declarativo mientras llega el módulo  
Build genera chunk separado validando el split  
 Chunk Reportes: **2.94 kB** · comprimido **1.26 kB**  
 Bundle principal: **292.23 kB** · **90.91 kB** gzip

## R13 — use(promise)

### Perfil con Suspense granular

Perfil consume Promises directamente con `use()`  
Mascotas e historial tienen boundaries **independientes**  
Un fallo parcial no bloquea el resto del perfil  
Cache del mock devuelve Promise estable, evita bucles  
 Diseño de datos: **estabilidad e invalidación** de Promises



#### APRENDIZAJE CLAVE

**Suspense modela el loading como una frontera declarativa.** La estabilidad e invalidación de la Promise forman parte del diseño de datos, no del componente visual.



# Actions de React 19 y experiencia inmediata

VETLAB ENGINEERING REPORT / REACT 19 CORE

## useActionState

Orquesta mutaciones de formulario, manejando estados de **pending**, errores y éxito de forma declarativa sin refetching manual.

MANEJO DE MUTACIÓN

## useOptimistic

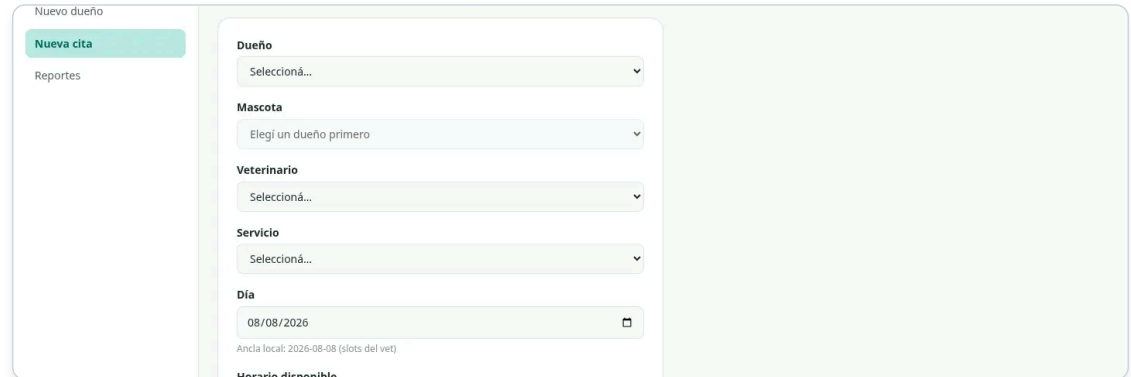
Actualiza la interfaz **instantáneamente** asumiendo éxito. Si la acción del servidor falla, React realiza un rollback automático.

ZERO LATENCY UI

## useFormStatus

Permite a componentes hijos descubrir si el formulario padre se está enviando, eliminando el **prop drilling** de estados de carga.

CONTEXTUAL PENDING



## Simulador de Actions

Estado: Pendiente

Mascota:

Luna (Golden Retriever)

Veterinario:

Dr. Mario Casas

Servicio:

Vacunación Anual

Presiona el botón para ver la actualización optimista inmediata vs la confirmación del servidor.

Confirmar Cita



# Sistema Visual, Accesibilidad y Mantenibilidad

VETLAB ENGINEERING REPORT / 2026



## Arquitectura Visual

### Design Tokens

Centralización de colores, tipografía y espaciado en variables CSS para coherencia global.

COLORES

SOMBRAS

RADIOS

### CSS Modules

Estilos co-ubicados por componente con scoping local automático para evitar colisiones.

### Theming Nativo

Soporte para modo oscuro mediante redefinición de tokens en el atributo data-theme.



## Accesibilidad Core

### Semántica HTML5

Uso de etiquetas estructurales y roles ARIA para mejorar la navegación con lectores de pantalla.

### Manejo de Foco

Control programático del foco en modales, trampas de teclado y restauración al cerrar overlays.

### Feedback de Estado

Comunicación clara de estados de carga, error y éxito mediante indicadores visuales y textuales.



## Solidez Técnica

### TypeScript Strict

Configuración estricta para garantizar seguridad en tiempo de compilación y evitar errores comunes.

### Modelos de Dominio

Contratos explícitos para entidades (Mascota, Cita, Veterinario) que guían el desarrollo.

### Cero Uso de "any"

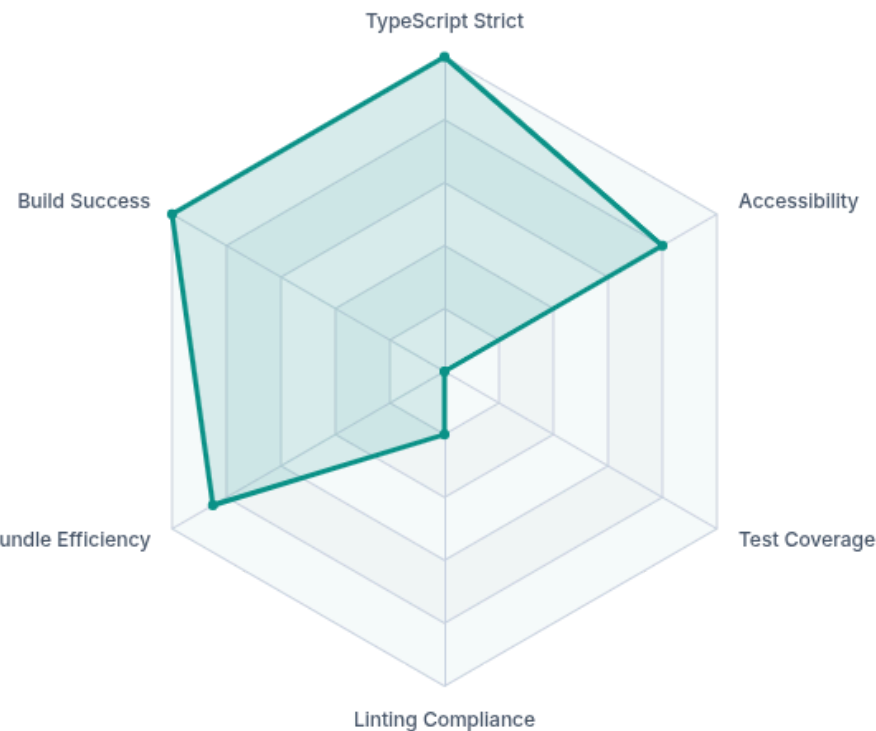
Integridad de tipos mantenida en todo el directorio src/ mediante tipado fuerte y genéricos.

✓ **Calidad de Ingeniería:** Cumplimiento del 100% en tipado estricto y estándares de diseño.

Modo: **Mantenibilidad Escalable**

# Evidencia de Calidad y Hallazgos

VETLAB ENGINEERING REPORT / 2024



Índice de salud técnica basado en verificación estática y métricas de build

## Verificación de Producción

|                                                              |                 |
|--------------------------------------------------------------|-----------------|
| > npm run build                                              | EXITOSO         |
| npm run typecheck                                            | SIN ERRORES     |
| Módulos Vite                                                 | 66              |
| Bundle Principal                                             | 90.91 kB (Gzip) |
| Code Splitting: OK   Reports Chunk: 2.94 kB   SRC: 6,693 LOC |                 |

## Hallazgos Técnicos

|                    |    |
|--------------------|----|
| Errores de ESLint  | 30 |
| Cobertura de Tests | 0% |

**Problema de Tooling:** La configuración de linting aplica el parser JS a archivos TSX/TS, impidiendo el control estático operativo a pesar de un typecheck correcto.

**Tests Automatizados:** Ausencia de archivos de prueba en el repositorio. Requiere estrategia de unit e integración.

# Aprendizajes finales y próximos pasos

VETLAB ENGINEERING REPORT / 2024

## Consolidación Técnica

### CORE REACT 19 Estado y Mutaciones

Dominio de Actions, useOptimistic y useActionState para flujos de datos asíncronos y robustos.

### ARQUITECTURA Contratos de Dominio

Separación estricta de intereses mediante TypeScript Strict y modelos de negocio centralizados.

### PERFORMANCE UX Concurrente

Uso eficiente de Suspense, useTransition y code-splitting para interfaces fluidas y escalables.

### VISUAL SYSTEM Tokens y Accesibilidad

Creación de un sistema de diseño mantenible con CSS Modules y semántica ARIA integrada.

"VetLab: De reglas de negocio a una arquitectura React explicable y extensible."

## Hoja de Ruta

### CALIDAD Tests Automatizados

Implementación de Vitest para lógica de dominio y Playwright para flujos críticos (E2E).

### INFRAESTRUCTURA Backend y CI/CD

Sustitución del Mock API por persistencia real y automatización de despliegue continuo.

### SEGURIDAD Auth y Permisos

Incorporación de gestión de sesiones, roles de usuario y protección de rutas sensibles.

### TOOLING Estandarización

Corrección de la configuración de ESLint para soporte completo de TypeScript y ESM.

Próximo hito: Beta operativa con persistencia real.